

Healthcare Provider Fraud Detection Data Cleaning Process

Author: Artur Melnyk
Updated: 2026-05-11
Version: v2.1
(Professional Edition)

Abstract

This document provides the technical documentation for the data cleaning and preparation phase of the Healthcare Provider Fraud Detection project. The process ensures that all raw healthcare datasets are clean, validated, and ready for exploratory analysis and predictive modeling. Emphasis was placed on reproducibility, data integrity, and methodological transparency.

[Dataset Source: Medicare Provider Fraud Detection on Kaggle](#)

Contents

1.	Introduction.....	2
2.	Loading Data	2
3.	Handling Missing Values.....	2
4.	Handling Outliers	3
9.	Verification.....	4
10.	Challenges and Lessons Learned	4
11.	Recommendations	5
12.	Conclusion	5
13.	Detailed Process Overview	5
14.	Version History	5
15.	Next Steps	5

Healthcare Provider Fraud Detection Data Cleaning Process

Author: Artur Melnyk

Updated: 2026-05-11

Version: v2.1

(Professional Edition)

1. Introduction

The purpose of this document is to detail the full data cleaning workflow applied to the *Medicare Provider Fraud Detection* dataset from Kaggle. The project aims to identify and mitigate healthcare provider fraud by ensuring the training and testing datasets are well-structured and consistent. The process included missing value imputation, data type alignment, referential integrity checks, and outlier control while preserving row counts and feature integrity.

Why Data Cleaning Matters

In healthcare fraud analytics, data quality directly influences model accuracy, compliance assurance, and cost savings. Clean, validated data prevents misclassification, supports fair provider assessment, and strengthens downstream model interpretability.

2. Loading Data

Eight datasets were imported using pandas from raw CSV files located in the `/data/raw` directory:

Dataset Category	Train	Test
Beneficiary Data	train_beneficiary.csv	test_beneficiary.csv
Inpatient Claims	train_inpatient.csv	test_inpatient.csv
Outpatient Claims	train_outpatient.csv	test_outpatient.csv
Feature Tables	train_features.csv	test_features.csv

Each dataset was loaded using the following command:

```
import pandas as pd

train_beneficiary = pd.read_csv('../data/raw/train_beneficiary.csv')
```

3. Handling Missing Values

Different imputation strategies were applied based on data type:

Data Type	Method	Example
Dates	Parsed using <code>errors='coerce'</code> ; missing → NaT	<code>df['DOB'] = pd.to_datetime(df['DOB'], errors='coerce')</code>
Numerical	Median imputation	<code>df[numerical_cols] = df[numerical_cols].fillna(df[numerical_cols].median())</code>
Categorical	Mode imputation	<code>df[categorical_cols] = df[categorical_cols].fillna(df[categorical_cols].mode().iloc[0])</code>
Fully Missing Columns	Preserved for structural consistency	N/A

This ensured logical handling of incomplete patient and claim records without arbitrary deletion.

Healthcare Provider Fraud Detection Data Cleaning Process

Author: Artur Melnyk

Updated: 2026-05-11

Version: v2.1

(Professional Edition)

4. Handling Outliers

Outliers were initially removed using row-wise IQR filtering, which caused significant row loss. This was replaced by a winsorization approach, which caps extreme values while preserving dataset size. Only columns with sufficient variance and more than 1000 non-null values were considered for winsorization.

```
def winsorize_column(series):
    Q1 = series.quantile(0.25)
    Q3 = series.quantile(0.75)
    IQR = Q3 - Q1
    return series.clip(Q1 - 1.5*IQR, Q3 + 1.5*IQR)
```

5. General Data Cleaning

Generic cleaning applied across datasets:

```
def data_cleaning_general(df):
    for col in df.select_dtypes(include=['float64', 'int64']).columns:
        if df[col].notnull().sum() > 1000 and df[col].std() > 0:
            df[col] = winsorize_column(df[col])
    return df
```

This ensured numerical stability while preserving referential keys (e.g., BeneID, ClaimID).

6. Data Type Conversion

Date and categorical consistency were enforced for reproducibility.

```
def convert_date_columns(df, date_columns):
    for col in date_columns:
        df[col] = pd.to_datetime(df[col], errors='coerce')
    return df

def convert_to_category(df, categorical_columns):
    df[categorical_columns] = df[categorical_columns].astype('category')
    return df
```

7. Applying Data Cleaning

Example full pipeline:

```
train_beneficiary = add_engineered_features(data_cleaning_beneficiary(train_beneficiary))
train_inpatient = data_cleaning_general(train_inpatient)
train_outpatient = convert_to_category(train_outpatient, categorical_columns_outpatient)
```

Healthcare Provider Fraud Detection Data Cleaning Process

Author: Artur Melnyk

Updated: 2026-05-11

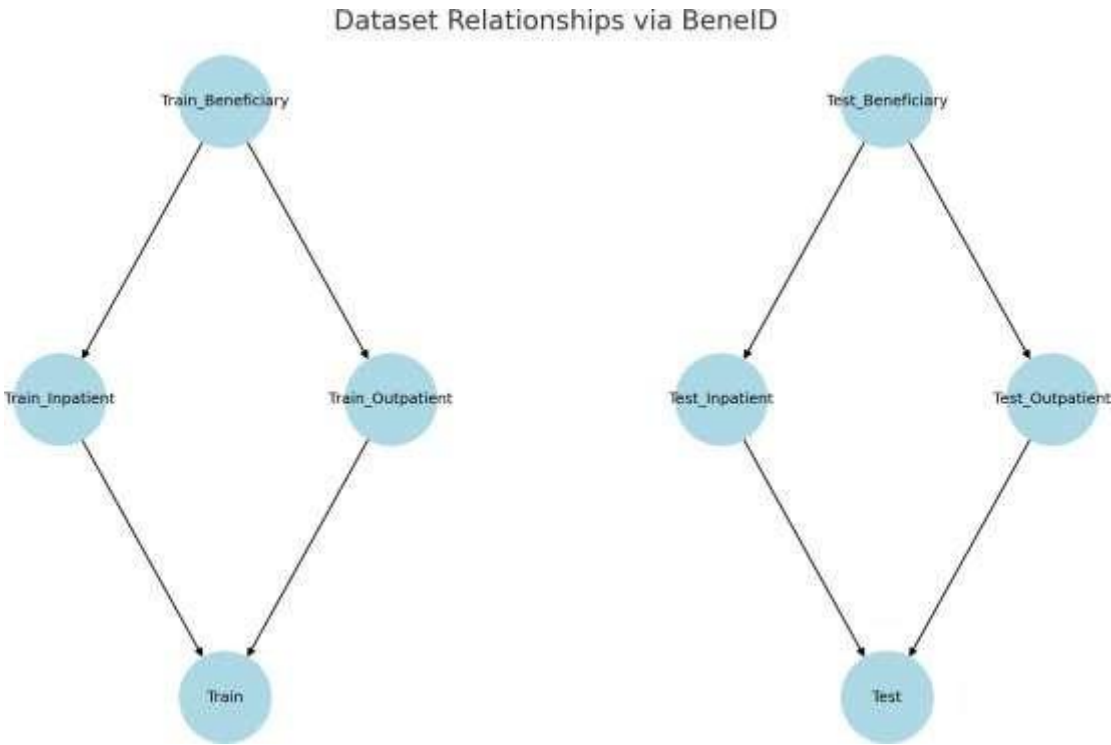
Version: v2.1

(Professional Edition)

This produced consistent, aligned datasets across all tables.

8. Dataset Relationship Diagram

Diagram showing relationships among datasets based on BenelD:



9. Verification

- No BenelD mismatches detected.
- Row counts preserved across all datasets.
- Train/Test datasets aligned structurally.
- Validation using:

```
print(df.isnull().sum().sum()) # 0 remaining NaNs
print(df.shape) # verified row consistency
```

10. Challenges and Lessons Learned

Category	Lesson	Outcome
Technical	IQR removal caused excessive data loss	Adopted winsorization for stability
Process	Sparse columns (e.g., CImProcedureCode_6) tempted deletion	Retained for potential modeling insight
Data Domain	Many missing DOD values are legitimate	Treated missingness as informative

Healthcare Provider Fraud Detection Data Cleaning Process

Author: Artur Melnyk

Updated: 2026-05-11

Version: v2.1

(Professional Edition)

11. Recommendations

- Drop or combine sparse features before modeling
- Create new features like total claims per provider, or diagnosis code frequency
- One-hot encode or label encode categoricals
- Apply class balancing or reweighting if PotentialFraud is imbalanced

12. Conclusion

All raw datasets have been cleaned, transformed, and validated. Outlier control is now non-destructive, types are correctly assigned, missing values handled, and the data is structurally aligned across train/test. The project is ready for EDA, modeling, or deployment pipelines. The cleaned datasets can also be exported locally in Parquet format for downstream AI-agent-based analytical applications. This document serves as the foundational reference for subsequent PRs: **feature/eda**, **feature/feature-eng**, and **feature/modeling**.

13. Detailed Process Overview

- Cleaned 8 raw datasets using pandas
- Filled missing dates, numerics, categoricals with appropriate logic
- Converted all date and categorical fields consistently
- Applied winsorization to numerical columns with enough data
- Added engineered feature AgeAtDeathOrLastClaim
- Verified dataset alignment and referential integrity
- Preserved row counts while removing numeric skew
- Used .info() and .isnull().sum() for structural validation

14. Version History

Version	Date	Changes
v1.0	2025-04-15	Initial data cleaning documentation
v2.0	2025-10-17	Professional Edition with formatting updates
v2.1	2026-05-11	Added local Parquet export support for downstream Healthcare AI Agent Integration

15. Next Steps

****Next Steps:****

This document concludes the data cleaning phase.

For follow-up work, see:

- PR #3 - feature/eda
- PR #4 - feature/feature-eng
- PR #5 - feature/modeling
- Export cleaned datasets locally as Parquet for downstream Healthcare AI Agent integration

Healthcare Data Cleaning Documentation v2.1 - May 2026

© [Artur Melnyk](#), MIT License (2025)